

StimGen 2 in five pages

Stimuli as sentences: a high-level description for reproducible cellular electrophysiology

Michele Giugliano

October 2026 — informal note, not for publication

Abstract

Electrophysiology experiments are defined by the stimuli they deliver, yet those stimuli are usually stored as sample files, scripts, or settings in acquisition software, and rarely in a form that can be read, checked, and regenerated years later. StimGen 2 describes a stimulus, or a whole protocol of trials, as a few lines of text with an exact meaning. One description can be rendered as samples at any rate. Its noise is reproducible from a seed. It produces files that a rig, a simulator, and an analysis script can all read. This note explains the idea, shows two short examples, and points to the tutorial and the full specification. Read it first.

1 The problem

A patch-clamp experiment on a single neuron is a dialogue. The experimenter injects a current (or imposes a voltage, or a conductance) and records the response. Most of what we learn depends on the *question* asked, i.e. on the stimulus: a family of current steps gives an f-I curve, a chirp gives an impedance profile, frozen noise gives spike-time reliability (Mainen and Sejnowski 1995), and a sweep of noise statistics gives a transfer function.

In practice the question is often recorded poorly. Stimuli live as arrays of samples exported from a script, as settings of a commercial acquisition program, or as a sequencer of files in a directory. Each form has a cost. An array of samples is opaque: we cannot see that it was “ten 2 s steps from 0 to 500 pA in shuffled order”. A script is readable only by its author and only while its dependencies exist. Program settings tie the experiment to one vendor. And when noise is involved, the realisation that was actually played is often lost, so the experiment cannot be repeated exactly or simulated faithfully.

These problems matter more now that experiments and models are run side by side. A model is fitted to recordings, then predicts the response to a new stimulus, which is then played to the same cell. If the experiment and the simulation do not receive *the same* stimulus, sample by sample, every comparison between them is in doubt. The FAIR principles ask that data be findable, accessible, interoperable, and reusable (Wilkinson et al. 2016). In electrophysiology, half of the data is the stimulus.

2 The idea: describe, then render

StimGen 2 separates *what* a stimulus is from *how* it is sampled. A stimulus is written as a short text with an exact mathematical meaning, independent of its duration, sampling rate, and D/A converter. It becomes samples only when it is used. The idea goes back to the stimulus descriptions of the earlier command-line tools of our group (Linaro et al. 2014). StimGen 2 makes it a language with a formal specification, a reference implementation, and a test suite.

The language has three layers, and each layer adds one idea.

1. **Waveform** (one channel). A sequence of segments, each a duration followed by an expression: 500ms dc(0), then 2s sine(100pA, 8Hz), and so on. Expressions combine sixteen generators (levels, ramps, relaxations, sines, square and saw waves, chirps, pulse trains, alpha and synaptic kernels, white, Ornstein–Uhlenbeck, and power-law noise, and samples from a file) with arithmetic and a few named functions (abs, clip, ...). Units are part of the numbers.
2. **Stimulus** (several channels on one clock). Named channels with units, for example a current command, a conductance, and digital trigger lines, plus time markers.

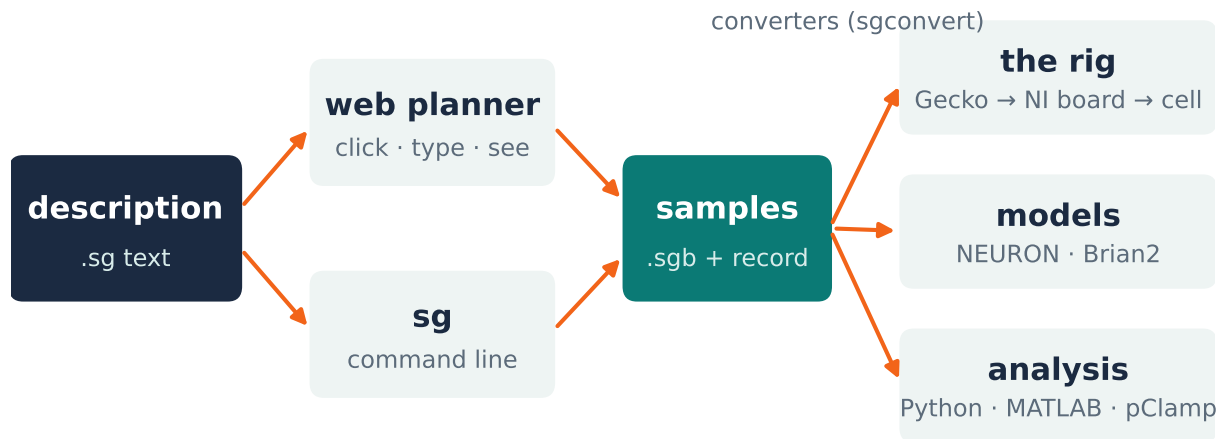


Figure 1: From a description to the rig, a model, or an analysis. The planner and `sg` produce the same samples. `sgconvert` turns them into the formats of the next program in the chain.

3. **Protocol** (many trials). Repetitions, parameter sweeps, ordering (sequential, grouped, shuffled), inter-trial periods, and the policy for noise across trials. This is the only layer with variables.

A protocol reads almost like the sentence a methods section would contain:

```

sg 2 protocol
sweep amp = from 0pA to 500pA step 50pA      # 11 conditions
order shuffled                                # in random, but recorded, order
repeat 3                                       # each condition three times
period 4s                                     # one trial every 4 s

stimulus {
  channel Iinj unit=pA {
    500ms dc(0)
    2s    dc($amp)
    500ms dc(0)
  }
}

```

The renderer expands it into 33 fully concrete trials. Each trial is itself a plain description, with `$amp` replaced by its value. The trials can be written out as a directory of `.sg` files, so the expanded form can be inspected, edited, or archived like any other set of files.

Three properties make the description reliable rather than merely compact.

Exact time. Durations and rates are held as exact integers and fractions, so segment boundaries fall on the same sample in every implementation. Rounding errors do not accumulate over long protocols.

Reproducible noise. Every noise term draws from its own stream of a counter-based random generator, Philox4x64-10 (Salmon et al. 2011). Each stream is keyed by a master seed and by the position of the term in the text. The same description, rate, and seed give the same samples on any machine. Writing `seed=21` inside a generator freezes that realisation wherever it appears. At the protocol level, one line chooses between noise that is fresh in every trial and noise that is frozen within a condition. If no seed is given, one is drawn from the operating system and recorded. The Ornstein–Uhlenbeck process uses the exact update (Gillespie 1996), so its statistics do not depend on the sampling rate.

Provenance travels with the samples. The output file (`.sgb`) is a short JSON header followed by little-endian 64-bit samples, channel after channel. The header holds the full description, the rate, the seed of every trial, the trial’s position in the protocol, and the version of the renderer. Any `.sgb` file can therefore be regenerated from its own header, and the header shows what the file contains without plotting it.

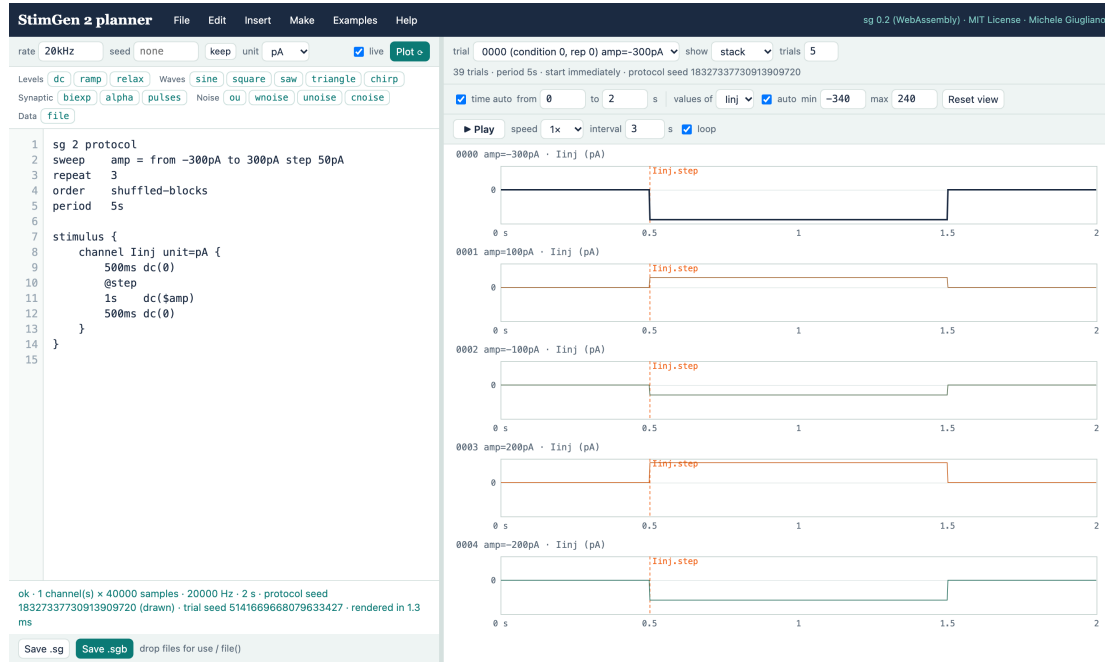


Figure 2: The planner showing a protocol: the description on the left, with buttons that insert generators, and the first five of its 39 trials (seven amplitudes, three repetitions, shuffled in blocks) stacked on the right. The trial menu, the axis limits, and the animation controls are above the plots.

3 Two ways in: the command line and the planner

The reference renderer, `sg`, is a single dependency-free C99 program. It checks, expands, and renders descriptions, and its built-in help documents every generator (`sg help ou`) and prints dozens of examples (`sg examples`):

```
sg check fi.sg # parse and validate; write nothing
sg render -r 20kHz -s 7 fi.sg # 11 trials: fi_sgb/0000.sgb ... 0010.sgb
sg render -r 10kHz -t -o - -e '1s sine(1, 5Hz) + ou(0, 0.2, 5ms)'
```

The *planner* is the same C code compiled to WebAssembly and wrapped in a web page (<https://blog.giuliano.info/StimGen2/>; Fig. 2). It needs no installation and nothing is sent to a server: the renderer runs inside the browser. We type a description on the left and see it plotted on the right as we type. Menus and a timeline editor build or edit segments without remembering the syntax, and the text is always updated with them. A protocol can be shown trial by trial, overlaid, stacked, or animated. A library of examples covers single waveforms and complete protocols. The planner saves the `.sg` text and the rendered `.sgb` samples, and these are identical to the files the command line writes for the same seed. An automatic test suite checks this against the command line.

The two entry points suit different moments of an experiment. The planner is for designing and checking a protocol, for teaching, and for showing a colleague what a stimulus does. The command line is for the rig, for scripts, and for batch rendering in a pipeline.

4 Example 1: an f-I curve, at the rig and in a model

The protocol of the previous section, without the repetitions and the shuffle, has eleven conditions. Rendered at 10 kHz, it yields eleven `.sgb` files of 3 s each (Fig. 3, left). The header of each file is enough to describe it in words:

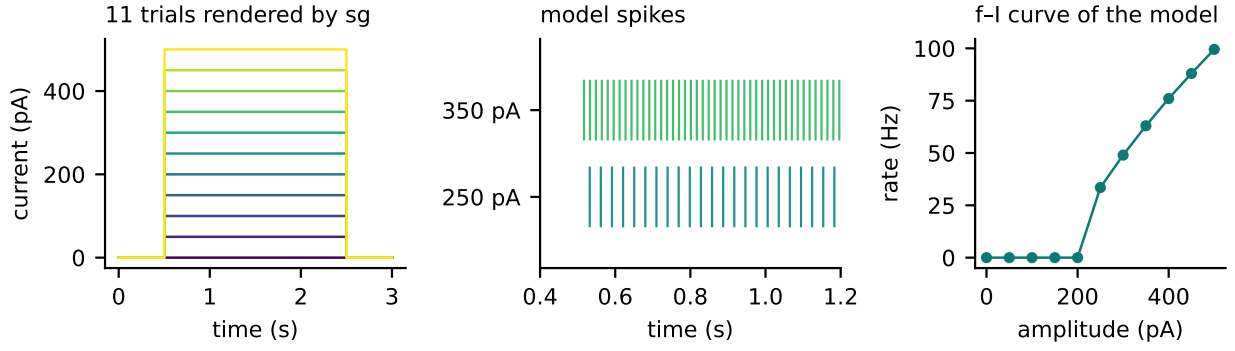


Figure 3: The f-I protocol. Left: the eleven trials rendered by `sg`. Middle: spikes of a model neuron driven by two of them. Right: the f-I curve of the model.

```
$ sgconvert info fi_sgb/0005.sgb
StimGen 2 sample file: 1 channel(s), 30000 samples at 10000 Hz (3 s); [...]
Protocol trial 5 (condition 5, repetition 0): amp=250pA.

Channel Iinj: min 0 pA, max 250 pA, mean 166.667 pA, SD 117.851 pA; rest 0 pA.
Made of 3 constant level(s):
    0 s to 0.5 s 0 pA
    0.5 s to 2.5 s 250 pA
    2.5 s to 3 s 0 pA
```

The same files go to a model. `sgconvert neuron` writes two-column files (time in ms, current in nA) for `Vector.play` in NEURON (Hines and Carnevale 1997), and `sgconvert brian` writes arrays for a `TimedArray` in Brian 2 (Stimberg et al. 2019). To keep this note self-contained, we fed the samples to a leaky integrate-and-fire neuron written in a few lines of NumPy ($\tau_m = 20$ ms, $R = 100$ M Ω , threshold -50 mV). The model starts firing at 250 pA and reaches 99.5 Hz at 500 pA (Fig. 3, right). A recording made with the same files can be put on the same axes without any bookkeeping: trial, amplitude, and timing come from the headers.

5 Example 2: frozen versus fresh noise, by changing one word

Spike-time reliability is measured by playing the same noisy current several times and comparing the spike trains (Mainen and Sejnowski 1995). A control experiment plays a *different* realisation each time. In StimGen 2 the two experiments differ by one word:

```
sg 2 protocol
repeat 10
noise per-condition # frozen: the same realisation in every trial
gap 1s # (per-trial: a fresh realisation in every trial)

stimulus {
    channel Iinj unit=pA { 2s ou(mean=150, sd=100, tau=5ms) }
}
```

Both protocols are fully reproducible. The fresh realisations are not “random” in the sense of being lost: each trial’s seed is derived from the protocol seed and recorded in its header. Driven by the frozen noise, the model neuron (with a small intrinsic noise of its own) fires reliably, with a reliability of 0.70 (the fraction of spikes matched within ± 2 ms across trials). With fresh noise, the reliability is 0.03 (Fig. 4). Because the realisations can be regenerated from the seeds, the *same* comparison can be repeated later in a detailed model, or in another cell, with exactly the same inputs.

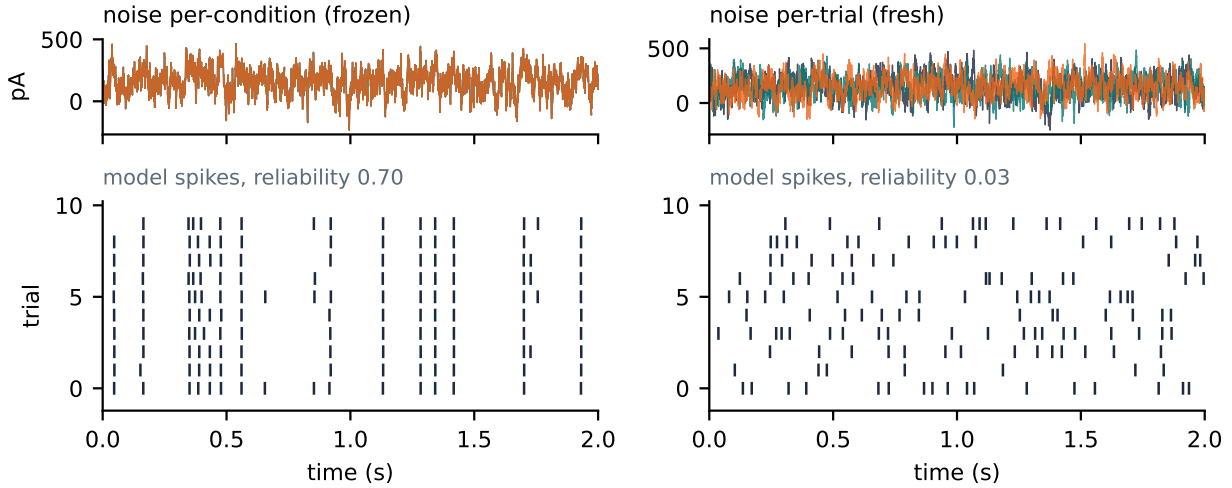


Figure 4: Frozen (left) and fresh (right) noise, rendered from protocols that differ by one word. Top: the ten trials overlaid. Bottom: spike rasters of the model neuron.

6 Formats and the next step

The `.sgb` file is the canonical output. It can be read in a few lines of C, Python, or Julia, as shown in the companion note on accessing `.sgb` files. For everything else, `sgconvert` turns one `.sgb` file into the format of the next program:

Table 1: Converters of `sgconvert` (one input file, one or more output files).

Command	Output	For
<code>info</code> , <code>json</code>	text description, header	humans, notebooks, lab books
<code>csv</code> , <code>npz</code> , <code>mat</code> , <code>h5</code>	time and channel arrays	Python, MATLAB, Julia, HDF5 tools
<code>atf</code>	Axon Text File	pClamp (episodic stimulation)
<code>neuron</code>	time (ms) and value (nA, mV, μ S)	NEURON <code>Vector.play</code>
<code>brian</code>	values and <code>dt</code>	Brian 2 <code>TimedArray</code>

Units are converted where a simulator expects its own (pA to nA, nS to μ S). Every converter keeps the seed and the description, so the provenance survives the conversion.

At the rig. In our laboratory the rendered files are played by *Gecko*, a separate command-line program that controls the National Instruments board and therefore the amplifier and the cell. StimGen 2 and Gecko are independent. StimGen 2 decides *what* is played and records it, and Gecko decides *how* it reaches the hardware. Any acquisition system that can read an array of samples, directly or through one of the converters above, can take Gecko’s place.

7 What this buys, and what it does not

For an experimentalist, StimGen 2 offers one short text per protocol, which can go in a lab book, an e-mail, or a methods section. That text regenerates every sample that was played, including the noise. For a modeller, it offers the stimuli of the experiment in the simulator’s own format, sample for sample. For both, the stimulus becomes part of the data that is shared and reused, not an undocumented detail of the setup.

There are limits. StimGen 2 describes stimuli and does not acquire data. It is *open-loop*: stimuli that depend on the cell’s response in real time (dynamic clamp, closed-loop control) belong to the real-time layer of the rig (Linaro et al. 2014). The renderer is a reference implementation that favours clarity over speed, although it renders five minutes of a two-channel noisy stimulus at 20 kHz in about half a second on a laptop. Elementary functions such as `sin` and `ln` may differ in their last bit between mathematical libraries, so reproducibility across platforms is guaranteed to within a few units in the last place (Level B of the specification). Bit-for-bit equality holds on any one platform.

8 Where to go next

- **Planner tutorial** (docs/slides/StimGen2-planner-tutorial.pptx): a step-by-step guide to the web planner, from the first step to a full protocol.
- **Command-line tutorial** (docs/slides/) and **sg examples**: the same ideas in a terminal.
- **Specification** (docs/build/stimgen2-spec.pdf): Part I is a tutorial, and Part II defines every generator, the sampling rules, the noise streams, the protocol layer, and the file format.
- **Source and planner**: <https://github.com/mgiugliano/StimGen2>, MIT licence.

References

- Gillespie, Daniel T. 1996. “Exact Numerical Simulation of the Ornstein-Uhlenbeck Process and Its Integral.” *Physical Review E* 54 (2): 2084–91. <https://doi.org/10.1103/PhysRevE.54.2084>.
- Hines, Michael L., and Nicholas T. Carnevale. 1997. “The NEURON Simulation Environment.” *Neural Computation* 9 (6): 1179–209. <https://doi.org/10.1162/neco.1997.9.6.1179>.
- Linaro, Daniele, João Couto, and Michele Giugliano. 2014. “Command-Line Cellular Electrophysiology for Conventional and Real-Time Closed-Loop Experiments.” *Journal of Neuroscience Methods* 230: 5–19. <https://doi.org/10.1016/j.jneumeth.2014.04.003>.
- Mainen, Zachary F., and Terrence J. Sejnowski. 1995. “Reliability of Spike Timing in Neocortical Neurons.” *Science* 268 (5216): 1503–6. <https://doi.org/10.1126/science.7770778>.
- Salmon, John K., Mark A. Moraes, Ron O. Dror, and David E. Shaw. 2011. “Parallel Random Numbers: As Easy as 1, 2, 3.” *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '11)*, 1–12. <https://doi.org/10.1145/2063384.2063405>.
- Stimberg, Marcel, Romain Brette, and Dan F. M. Goodman. 2019. “Brian 2, an Intuitive and Efficient Neural Simulator.” *eLife* 8: e47314. <https://doi.org/10.7554/eLife.47314>.
- Wilkinson, Mark D. et al. 2016. “The FAIR Guiding Principles for Scientific Data Management and Stewardship.” *Scientific Data* 3: 160018. <https://doi.org/10.1038/sdata.2016.18>.